

กลุ่มที่ 4 : CPU Process Scheduling ระบบจัดคิว Process ของ CPU

1. Problem Analysis

1.1 Problem

ระบบปฏิบัติการต้องจัดสรร CPU ให้กับ Process หลายตัวที่รอการประมวลผล โดยแต่ละ Process มีเวลาที่ต้องใช้ CPU แตกต่างกัน จึงต้องมี Algorithm สำหรับกำหนดลำดับ การใช้ CPU วิทยานี้ศึกษาการจัดคิว Process โดยเปรียบเทียบ First Come First Served (FCFS) และ Round Robin (RR)

Algorithm A: First Come First Served (FCFS) จัด Process ตามลำดับที่เข้ามาใน Queue โดย Process ที่เข้ามาก่อนจะได้รับ CPU ก่อน

Algorithm B: Round Robin (RR) แบ่งเวลา CPU ให้แต่ละ Process ตามลำดับ โดยกำหนด Time Quantum = 3 หาก Process ยังทำงานไม่เสร็จหลังใช้ CPU ครบ 3 หน่วยเวลา จะถูกนำกลับไปต่อท้าย Circular Queue

1.2 Input

Process	Arrival Time	Burst Time
P1	0	8
P2	0	4
P3	0	9
P4	0	5

Time Quantum ของ Round Robin = 3

1.3 Output

- ลำดับการทำงานของ Process
- Waiting Time ของแต่ละ Process
- Turnaround Time ของแต่ละ Process
- จำนวน Context Switch
- Average Waiting Time
- Average Turnaround Time
- ผลการเปรียบเทียบ FCFS และ Round Robin

1.4 Constraints

- มี Process ทั้งหมด 4 Process
- กำหนด Arrival Time = 0 สำหรับทุก Process
- Burst Time ของแต่ละ Process แตกต่างกันไป
- FCFS ทำงานตามลำดับ FIFO
- Round Robin ใช้ Circular Queue
- Time Quantum ของ Round Robin เท่ากับ 3
- Process ที่ยังทำงานไม่เสร็จจะถูกนำกลับไปต่อท้าย Queue

1.5 Queue เก็บข้อมูลอะไร

Queue เก็บข้อมูลของ Process ที่รอใช้ CPU ได้แก่ Process ID, Arrival Time, Burst Time และ Remaining Time

2. Queue Design

2.1 Algorithm A: FCFS

ใช้ Queue แบบ FIFO (First In First Out)

ลำดับเริ่มต้น: Front $\rightarrow$ P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ Rear

ลำดับการทำงาน: P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P4

2.2 Algorithm B: Round Robin

ใช้ Circular Queue และกำหนด Time Quantum = 3

เมื่อ Process ได้ CPU จะทำงานได้สูงสุด 3 หน่วยเวลา ถ้าทำงานเสร็จจะถูกนำออกจาก Queue

แต่ถ้ายังไม่เสร็จจะถูกนำกลับไปต่อท้าย Queue

3. Pseudocode Algorithm A — FCFS

Algorithm FCFS

1. Create an empty Queue
2. Enqueue P1, P2, P3, P4 into Queue
3. Set currentTime = 0
4. While Queue is not empty
 - 4.1 Dequeue the Process at the front
 - 4.2 Execute the Process until completion
 - 4.3 Calculate Completion Time
 - 4.4 Calculate Waiting Time
 - 4.5 Calculate Turnaround Time
5. Calculate Average Waiting Time
6. Calculate Average Turnaround Time
7. Count Context Switch
8. Display results

4. Pseudocode Algorithm B — Round Robin

Algorithm ROUND_ROBIN

1. Create an empty Circular Queue
2. Set Time Quantum = 3
3. Enqueue P1, P2, P3, P4
4. Set currentTime = 0
5. While Circular Queue is not empty
 - 5.1 Dequeue the Process at the front
 - 5.2 Execute Process for min(Time Quantum, Remaining Time)
 - 5.3 Update currentTime
 - 5.4 Update Remaining Time

5.5 If Remaining Time = 0

Process is completed

Calculate Completion Time

Else

Enqueue Process back into Circular Queue

6. Calculate Waiting Time

7. Calculate Turnaround Time

8. Count Context Switch

9. Calculate average values

10. Display results

5. Queue Trace

Round Robin, Time Quantum = 3

Step	Operation	Queue Before	CPU	Remaining หลังทำงาน	Queue After
1	ENQUEUE P1	[]	-	P1=8	[P1]
2	ENQUEUE P2	[P1]	-	P2=4	[P1,P2]
3	ENQUEUE P3	[P1,P2]	-	P3=9	[P1,P2,P3]
4	ENQUEUE P4	[P1,P2,P3]	-	P4=5	[P1,P2,P3,P4]
5	DEQUEUE P1	[P1,P2,P3,P4]	P1 ใช้ 3	P1=5	[P2,P3,P4,P1]
6	DEQUEUE P2	[P2,P3,P4,P1]	P2 ใช้ 3	P2=1	[P3,P4,P1,P2]
7	DEQUEUE P3	[P3,P4,P1,P2]	P3 ใช้ 3	P3=6	[P4,P1,P2,P3]
8	DEQUEUE P4	[P4,P1,P2,P3]	P4 ใช้ 3	P4=2	[P1,P2,P3,P4]
9	DEQUEUE P1	[P1,P2,P3,P4]	P1 ใช้ 3	P1=2	[P2,P3,P4,P1]
10	DEQUEUE P2	[P2,P3,P4,P1]	P2 ใช้ 1	P2=0	[P3,P4,P1]
11	DEQUEUE P3	[P3,P4,P1]	P3 ใช้ 3	P3=3	[P4,P1,P3]
12	DEQUEUE P4	[P4,P1,P3]	P4 ใช้ 2	P4=0	[P1,P3]
13	DEQUEUE P1	[P1,P3]	P1 ใช้ 2	P1=0	[P3]
14	DEQUEUE P3	[P3]	P3 ใช้ 3	P3=0	[]

ลำดับ CPU: P1 → P2 → P3 → P4 → P1 → P2 → P3 → P4 → P1 → P3

6. Correctness / Invariant

6.1 FCFS

Algorithm FCFS ถูกต้องเนื่องจากใช้หลัก FIFO ซึ่ง Process ที่เข้าคิวก่อนจะถูกนำออกจาก Queue ก่อน

ทำให้ลำดับการประมวลผลเป็นไปตามลำดับการเข้าคิว

Invariant: ก่อนการประมวลผลแต่ละรอบ Process ที่อยู่ด้านหน้า Queue คือ Process ที่เข้ามาก่อน Process อื่นทั้งหมดที่ยังไม่ได้รับการประมวลผล

6.2 Round Robin

Round Robin ถูกต้องเนื่องจากทุก Process จะได้รับโอกาสใช้ CPU ตามลำดับ และ Process ที่ยังไม่เสร็จ หลังครบ

Time Quantum จะถูกนำกลับไปท้าย Queue

Invariant: ก่อนการประมวลผลแต่ละรอบ Process ที่อยู่ด้านหน้า Circular Queue จะเป็น Process ที่รอการประมวลผลในรอบปัจจุบัน และ Process ที่ยังไม่เสร็จจะถูกนำกลับไปท้าย Queue

7. Time Complexity

Operation	FCFS	Round Robin
enqueue()	$O(1)$	$O(1)$
dequeue()	$O(1)$	$O(1)$
peek()	$O(1)$	$O(1)$
search()	$O(n)$	$O(n)$
display()	$O(n)$	$O(n)$

สำหรับการจัดการ Queue การ enqueue และ dequeue สามารถทำได้ใน $O(1)$ ส่วน search และ display

ต้องตรวจสอบสมาชิกของ Queue จึงเป็น $O(n)$

สำหรับ Round Robin จำนวนการทำงานขึ้นกับจำนวน Time Slice โดยเมื่อ Time Quantum เป็นค่าคงที่

จำนวนรอบจะแปรผันตามปริมาณ Burst Time ที่ต้องประมวลผล

8. Space Complexity

ทั้ง FCFS และ Round Robin ต้องเก็บ Process ที่รออยู่ใน Queue หากมี Process สูงสุด n รายการ Space

Complexity โดยทั่วไปเป็น $O(n)$ โดย Round Robin ยังต้องเก็บ Remaining Time ของแต่ละ Process

9. Java Program

```
import java.util.*;

class Process {
    String id;
    int arrivalTime;
    int burstTime;
    int remainingTime;
    int completionTime;
    int waitingTime;
    int turnaroundTime;

    Process(String id, int arrivalTime, int burstTime) {
        this.id = id;
        this.arrivalTime = arrivalTime;
        this.burstTime = burstTime;
        this.remainingTime = burstTime;
    }
}

public class CPUScheduling {
    static final int TIME_QUANTUM = 3;

    static Process[] createProcesses() {
        return new Process[] {
            new Process("P1", 0, 8),
            new Process("P2", 0, 4),
            new Process("P3", 0, 9),
            new Process("P4", 0, 5)
        };
    }

    static void fcfs(Process[] processes) {
        int currentTime = 0;
        int contextSwitch = 0;

        for (int i = 0; i < processes.length; i++) {
```

```

    Process p = processes[i];
    currentTime = Math.max(currentTime, p.arrivalTime);

    currentTime += p.burstTime;
    p.completionTime = currentTime;
    p.turnaroundTime = p.completionTime - p.arrivalTime;
    p.waitingTime = p.turnaroundTime - p.burstTime;

    if (i > 0) contextSwitch++;
}

printResults(processes, contextSwitch);
}

static void roundRobin(Process[] processes) {
    Queue<Process> queue = new ArrayDeque<>();
    for (Process p : processes) queue.offer(p);

    int currentTime = 0;
    int contextSwitch = 0;
    Process previous = null;

    while (!queue.isEmpty()) {
        Process p = queue.poll();

        if (previous != null && previous != p)
            contextSwitch++;

        int time = Math.min(TIME_QUANTUM, p.remainingTime);
        currentTime += time;
        p.remainingTime -= time;

        if (p.remainingTime == 0) {
            p.completionTime = currentTime;
            p.turnaroundTime = p.completionTime - p.arrivalTime;
            p.waitingTime = p.turnaroundTime - p.burstTime;
        } else {
            queue.offer(p);
        }
    }
}

```

```

    }

    previous = p;
}

printResults(processes, contextSwitch);
}

static void printResults(Process[] processes, int contextSwitch) {
    double totalWaiting = 0;
    double totalTurnaround = 0;

    for (Process p : processes) {
        System.out.println(
            p.id + " Waiting=" + p.waitingTime +
            " Turnaround=" + p.turnaroundTime
        );
        totalWaiting += p.waitingTime;
        totalTurnaround += p.turnaroundTime;
    }

    System.out.println("Average Waiting Time = "
        + totalWaiting / processes.length);
    System.out.println("Average Turnaround Time = "
        + totalTurnaround / processes.length);
    System.out.println("Context Switches = " + contextSwitch);
}

public static void main(String[] args) {
    fcfs(createProcesses());
    roundRobin(createProcesses());
}
}

```


10. Test Cases

Test Case	Input	Expected Result
1. Normal Case	P1=8, P2=4, P3=9, P4=5	Process ทั้งหมดทำงานเสร็จ
2. Empty Queue	ไม่มี Process	ระบบแจ้งว่า Queue ว่าง
3. Single Item	P1=8	P1 ทำงานจนเสร็จ
4. Large Queue	Process จำนวนมาก	ระบบสามารถประมวลผล Process ได้ทั้งหมด
5. Special / Edge Case	Burst Time เท่ากันหลาย Process	Process ทำงานตามกฎของ Algorithm
6. Cancel Case	ยกเลิก Process ที่อยู่ใน Queue	Process ที่ถูกยกเลิกจะไม่ถูกประมวลผล

11. Experimental Comparison

การทดลองต้องใช้ขนาดข้อมูล $n = 100, 1,000, 10,000$ และ $50,000$ วัด Execution Time ด้วย `nanoTime()` มี warm-up ใช้ seed คงที่ และเฉลี่ยผลจาก 5 รอบ

n	FCFS Average Time (ns)	Round Robin Average Time (ns)
100	30,220.00	67,080.00
1,000	120,660.00	410,360.00
10,000	516,020.00	701,480.00
50,000	744,560.00	1,944,600.00

สรุปผลการทดลอง

จากผลการทดลองพบว่า เมื่อจำนวน Process เพิ่มขึ้น เวลาที่ใช้ในการประมวลผลของทั้ง FCFS และ Round Robin มีแนวโน้มเพิ่มขึ้น โดยจากการทดลองชุดนี้ FCFS ใช้เวลาในการประมวลผลน้อยกว่า Round Robin ในทุกขนาดข้อมูลที่ทดสอบ เนื่องจาก FCFS ประมวลผลแต่ละ Process จนเสร็จแล้วจึงไปยัง Process ถัดไป ขณะที่ Round Robin มีการแบ่ง CPU ตาม Time Quantum = 3 และนำ Process ที่ยังทำงานไม่เสร็จกลับเข้าสู่ Queue ทำให้มีการดำเนินการกับ Queue หลายครั้งกว่า

12. สรุปเปรียบเทียบ Algorithm A และ B

ประเด็น	FCFS	Round Robin
Data Structure	Queue	Circular Queue
หลักการ	มาก่อนได้ก่อน	แบ่งเวลา CPU ตาม Time Quantum
Enqueue	$O(1)$	$O(1)$
Dequeue	$O(1)$	$O(1)$
Average Waiting Time	10.25	15.00
Average Turnaround Time	16.75	21.50
Context Switch	3	9
Fairness	น้อยกว่าเมื่อมีงานแรกใช้เวลานาน	สูงกว่าเพราะแบ่งเวลาให้แต่ละ Process
ข้อดี	ง่ายและมี Context Switch น้อย	เหมาะกับการให้หลาย Process ได้รับ CPU อย่างทั่วถึง
ข้อจำกัด	Process ที่อยู่หลังงานยาวอาจรอนาน	มี Context Switch มากกว่า
เหมาะกับกรณี	Batch Processing	Interactive System

จากชุดข้อมูลที่กำหนด FCFS มี Average Waiting Time และ Average Turnaround Time ต่ำกว่า Round Robin แต่ Round Robin มีความยุติธรรมในการแบ่งเวลา CPU มากกว่า และเหมาะกับ Interactive System ที่ต้องการให้ผู้ใช้งานได้รับการตอบสนองอย่างต่อเนื่อง